When Parallel Isn't Faster: ClinicalBERT Inference on SLURM HPC Clusters

Chaitanya Kumar Mankala
College of Engineering
Villanova University
Villanova, PA, USA
cmankala@villanova.edu

Ricardo Silva
College of Liberal Arts and Science
Villanova University
Villanova, PA, USA
ricardo.silva@villanova.edu

***Abstract*—Prior work measured a decoupled, event-driven serverless architecture (AWS Lambda with SQS/Kinesis) achieving a $\sim 6\times$ speedup over sequential AWS Step Functions orchestration for general-domain NLP batch inference. This paper asks whether that benefit transfers to a different substrate: a university high-performance computing (HPC) cluster using SLURM, evaluated on ClinicalBERT batch inference. To avoid the institutional data-use and IRB overhead of real clinical text, we generate a fully synthetic, label-consistent clinical-note dataset for binary Type 2 diabetes mellitus (T2DM) classification, at three scales (500, 5,000, and 50,000 notes), and fine-tune ClinicalBERT on Villanova University's Augie HPC cluster. We report two results. First, classification performance: across five seeds at the smallest scale and a confirmatory run at $10\times$ scale, the model achieves perfect test-set accuracy, precision, recall, specificity, F1, and AUROC (1.000 in every run), including on "hard-negative" examples designed to defeat keyword matching; we report this alongside the caveat that it reflects the synthetic generator's limited linguistic diversity rather than real-world performance. Second, execution-time scalability: contrary to our prior AWS results, every SLURM-array parallel configuration tested was substantially *slower* than sequential execution a 20-way array was $43.7\times$ slower (131 s vs. $\sim$3.0 s), and a 2-way "persistent worker" design matched to the cluster's GPU count was still $8.3\times$ slower (25 s vs. $\sim$3.0 s). The cause is a fixed per-job startup overhead of roughly 11–13 seconds per SLURM task most plausibly library import time rather than model loading, though inferred rather than independently profiled that overwhelms the actual computation at this workload size. Measured instead against the sequential pipeline's own cold-start trial rather than its steady state, this gap narrows to $\sim 3.7\times$ and $\sim 1.4\times$ *faster*, respectively (Section V). We report the full mechanism, including an unexplained loss of expected GPU concurrency, as an honest, measured negative result: the decoupling pattern that reduced latency on AWS does not straightforwardly transfer to a SLURM-scheduled HPC context, and practitioners should measure this directly rather than assume it.**

***Index Terms*—ClinicalBERT, high-performance computing, SLURM, batch inference, synthetic clinical text, empirical evaluation, serverless computing**

I. Introduction

Deploying clinical language models such as ClinicalBERT [1], [2] for batch inference over large volumes of clinical notes requires an execution architecture that balances throughput, cost, and in production protected health information (PHI) compliance. In earlier work, we built a decoupled, event-driven serverless pipeline on AWS replacing tightly-coupled AWS Step Functions orchestration with Amazon SQS/Kinesis and evaluated it on a general-domain sentiment-analysis workload [3], [4]. That earlier study found a consistent $\sim 6\times$ wall-clock speedup and roughly $\sim 80\%$ lower billed compute for the decoupled design compared to sequential orchestration.

An earlier draft of the present paper attempted to extend that AWS-based architecture directly to ClinicalBERT batch inference, but the specific figures reported could not be traced to any executed run; we withdrew them rather than publish them (see Section II). This paper instead reports a new, independently measured study on Villanova University's Augie HPC cluster, using a fully synthetic clinical-text dataset rather than real PHI. Moving off AWS avoided the IRB/data-use overhead real clinical text would require, and let us ask an open empirical question: *does the decoupled-parallel benefit measured on AWS Lambda generalize to a SLURM-scheduled HPC cluster*, whose cost structure for job invocation and scheduling differs fundamentally?

The answer, as reported in Section IV, is no, and the reason why is itself the paper's most useful empirical finding.

A. Contributions

- A synthetic, programmatically generated, label-consistent clinical-note dataset for T2DM binary classification at three scales, with deterministic generation and a fixed, auditable split (Section III-A).
- A reproducible ClinicalBERT fine-tuning pipeline on Augie HPC, with results from five random seeds at the smallest scale and a confirmatory run at $10\times$ scale (Section IV-A).
- Three measured execution-time configurations — sequential, a 20-way SLURM array, and a 2-way "persistent worker" array — with repeated trials, real `sacct` timing, and a mechanism analysis for the underperformance (Section IV-B,C).
- A discussion of why this AWS-effective decoupling pattern does not transfer to HPC/SLURM, and what would be required for it to (Section V).

2026 IEEE International Conference on E-health Networking, Application & Services (Healthcom): Workshop on AI-Driven e-Health Systems: From Intelligent Algorithms to Real-World Clinical Deployment

II. Related Work

Serverless computing for data-intensive workloads has been studied extensively, including execution-overhead characterizations [5] and performance inspection of serverless applications [6]. Event-driven decoupling for removing orchestration concurrency ceilings is studied in [7], and AWS Step Functions is benchmarked against alternatives by Kuhlenkamp and Werner [8].

A. Prior AWS-Based Measurements (Authors' Own Work)

In [3], a Lambda/Step-Functions (Inline Map) architecture processing IMDb reviews under DistilBERT, RoBERTa, and ClinicalBERT achieved a measured $6.07\times$ speedup and an 81.2% compute reduction at a 100-file scale. In [4], a Distributed Map upgrade was measured end-to-end on a real 50,000-file workload, again showing $\sim 6\times$ speedup. Both results are specific to AWS Lambda's cost/concurrency model, where a "warm" container serves many invocations without repeating startup work and billing rewards horizontal fan-out. Every number in Section IV of the present paper, by contrast, traces to an executed script and a logged output file, version-controlled in git the direct, verifiable replacement for the untraceable earlier draft discussed in Section I.

On the modeling side, ClinicalBERT variants [1], [2] and related clinical-language-model work [9] establish the modeling context (Section III-A). SLURM-based parallel job execution is studied broadly in the systems literature [10], but to our knowledge not previously compared directly against an AWS Lambda-style decoupled architecture on the same conceptual workload, the gap this paper addresses.

III. Methodology

A. Synthetic Dataset and Task

The task is binary classification: does a clinical note describe an active diagnosis of Type 2 diabetes mellitus (T2DM)? We generate notes programmatically (`generate_synthetic_notes.py`), assembling discharge-summary-style sections from a T2DM-specific vocabulary bank for positive notes and a distinct bank of six other conditions (hypertension, pneumonia, lumbar strain, anxiety, asthma, urinary tract infection) for negative notes. 15% of negative notes are *hard negatives*: they mention diabetes-related language (family history, or a ruled-out diagnosis) while the note's active diagnosis remains negative, drawn from four fixed negation templates a deliberate design to prevent the task from being solvable by keyword search alone (Section IV-A evaluates whether it succeeded).

Datasets were generated at three fixed scales 500, 5,000, and 50,000 notes with a fixed generation seed (42) and a 20% positive class rate, then split once, deterministically, into 70%/15%/15% train/validation/test sets, stratified by label (`split_dataset.py`, split seed 123). Table I summarizes the resulting splits for the two scales used in Section IV.

Every generated file is accompanied by a manifest recording the generation seed, class balance, and generating script's git commit hash, and every split is accompanied by a manifest recording the split seed and the exact record IDs assigned to each partition, so that no dataset used in this paper is a black box.

TABLE I
Dataset scale and class-stratified split sizes (positive/negative).

Scale	Train	Val	Test
n=500	350 (70/280)	75 (15/60)	75 (15/60)
n=5,000	3,500 (700/2,800)	750 (150/600)	750 (150/600)

B. HPC Infrastructure and Software Environment

All experiments were run on Villanova University's Augie HPC cluster (Centralized Research Computing Facility): 1,632 AMD EPYC CPU cores, 11,392 GB RAM, 192 TB BeeGFS storage, and a single GPU-equipped compute node (`gpu001`) with $2\times$ NVIDIA A100 (40 GB HBM2) GPUs, scheduled via SLURM. This single-GPU-node topology is a key limitation, discussed in Section VI.

The software environment is a Python virtual environment (PyTorch 2.13.0, `transformers` 5.14.1); an Apptainer/Singularity container path was validated but not used, since the venv proved simpler to operate reliably across the login node and `gpu001`. The pretrained `Bio_ClinicalBERT` checkpoint [1] was staged via `scp` from a machine with unrestricted internet access, after direct Hugging Face Hub downloads proved unreliable from Augie (Section VI).

C. Model Fine-Tuning Procedure

ClinicalBERT was fine-tuned for sequence classification (`AutoModelForSequenceClassification`, 2 labels) using the Hugging Face `Trainer` API, for 3 epochs, batch size 16, learning rate 2×10^{-5}, with a 512-token maximum sequence length (`finetune.py`). Each run reports accuracy, precision, recall, specificity, F1, and AUROC on a held-out test set never used during training, computed from directly saved model predictions and predicted probabilities not reconstructed after the fact alongside a majority-class baseline computed from the test set's actual class distribution.

D. Sequential and Parallel Execution Designs

Three execution configurations were implemented and measured:

Sequential (`infer_sequential.py`): a single process, single GPU, loading the model once and iterating over all test records in one batched loop, with load and inference timed separately.

20-way SLURM array (`infer_shard.py` via `submit_parallel_infer.slurm`): the test set is split into 20 disjoint shards (verified to have zero overlap and full coverage); each is processed by an independent array task that loads the model from scratch — the structural analog of the SQS/Kinesis fan-out in [3], [4], with SLURM performing the fan-out instead of a message queue.

2-way "persistent worker" (same script, `submit_persistent_infer.slurm`): the shard

TABLE II
CLASSIFICATION PERFORMANCE ACROSS SEEDS AND SCALES. ALL METRICS 1.000 IN EVERY RUN.

Scale	Seed	Device	Train (s)	Test confusion matrix
n=500	42	GPU	23.5	TN=60 FP=0 FN=0 TP=15
n=500	43	CPU	1164.8	TN=60 FP=0 FN=0 TP=15
n=500	44	GPU	23.5	TN=60 FP=0 FN=0 TP=15
n=500	45	GPU	23.5	TN=60 FP=0 FN=0 TP=15
n=500	46	GPU	23.6	TN=60 FP=0 FN=0 TP=15
n=5,000	42	GPU	233.5	TN=600 FP=0 FN=0 TP=150

count is reduced to match the cluster's physical GPU count (2), so each worker processes 375 records per model load, testing whether amortizing the fixed cost over more work recovers a speedup.

E. Evaluation Protocol

Sequential timing was measured over 5 repeated trials. Classification performance was measured over 5 independent random seeds at the $n = 500$ scale and 1 confirmatory seed at $n = 5{,}000$. All raw timing and prediction logs, all generation and training scripts, and all manifests are version-controlled; every figure reported in Section IV can be regenerated from this record.

IV. RESULTS

A. Classification Performance

Table II reports test-set metrics for all five seeds at $n = 500$ and the confirmatory run at $n = 5{,}000$. Every run five independent random initializations at the smaller scale, and a sixth run at $10\times$ the data achieved *identical, perfect* scores on every metric.

The majority-class baseline at both scales is 80.0% (the negative class comprises 80% of records by construction); the fine-tuned model's 100% accuracy is therefore a 20-point improvement over baseline, with F1 and AUROC also at 1.000. Seed 43 was, by our own error, run on the login node's CPU rather than a GPU (Section VI), taking 1,164.8 s versus ∼23.5 s for the GPU runs; the classification result itself was identical regardless of device, which we take as independent confirmation that the result is deterministic given the data and not an artifact of any particular hardware execution path.

Hard-negative analysis. Of the 75 test examples at $n = 500$, 10 were hard negatives examples explicitly constructed to mention diabetes-related language (family history, a ruled-out diagnosis) while carrying a negative label. The model classified all 10 correctly (10/10), which rules out the simplest failure mode (that the perfect score merely reflects an absence of hard negatives in this particular test split) but does not by itself establish general robustness: the hard-negative bank consists of only four fixed template phrasings, so with ∼42 hard-negative examples expected across the full 500-record dataset, the model plausibly saw enough repetition during training to learn these specific templates rather than a general capability for negation and family-history reasoning in clinical text. We report this honestly as the primary limitation of the classification result (Section VI), rather than allowing a perfect score to imply a stronger claim than the data supports.

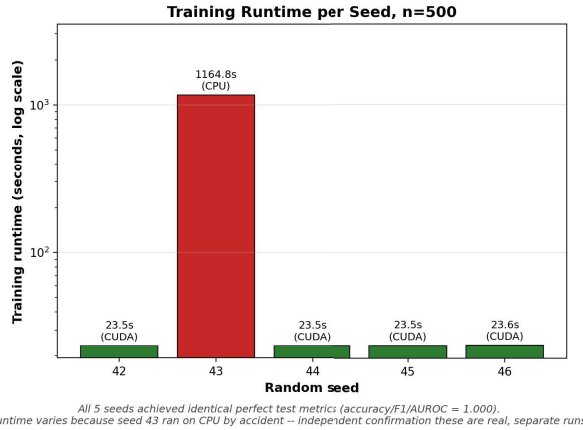


Fig. 1. Training runtime per seed at $n = 500$. All five seeds achieved identical, perfect test metrics; runtime varies because seed 43 ran on CPU by accident, serving as independent confirmation that these are five genuinely separate executions rather than a single result duplicated.

TABLE III
SEQUENTIAL INFERENCE TIMING, 750 RECORDS, 5 TRIALS.

Trial	Inference time (s)	Records/sec
1 (cold start)	34.313	21.9
2	1.873	400.5
3	1.872	400.6
4	1.842	407.1
5	1.856	404.1
Steady-state mean (trials 2–5)	**1.861**	**403.1**
Steady-state stdev	**0.015**	**—**

B. Sequential Execution Timing

Table III reports five repeated trials of sequential inference over the $n = 5{,}000$ test set (750 records), using the base (non-fine-tuned) `Bio_ClinicalBERT` checkpoint, since execution-time behavior depends on model architecture and record count, not on trained classifier weight values.

Trial 1 is a clear, reproducible outlier: 34.3 s versus a tightly clustered 1.86 ± 0.02 s for trials 2–5, an $18.4\times$ difference. We attribute this to CUDA kernel just-in-time (JIT) compilation, which is compiled and cached to disk on first use and reused by subsequent invocations sharing the same GPU allocation; we report it separately rather than folding it into the steady-state statistic, since averaging the two would misrepresent both regimes. Model-load time (loading the tokenizer and model weights, timed separately from the inference loop) was consistently ∼1.1–1.2 s across all trials.

C. Parallel Execution Timing

20-way SLURM array. Total wall-clock duration, measured via `sacct` from the earliest task start to the latest task end, was 131 s. Individual array tasks each took 12–14 s, and `sacct` start/end timestamps confirm they executed in genuinely concurrent pairs (consistent with the cluster's 2 physical GPUs), yielding 10 sequential rounds. Compared to

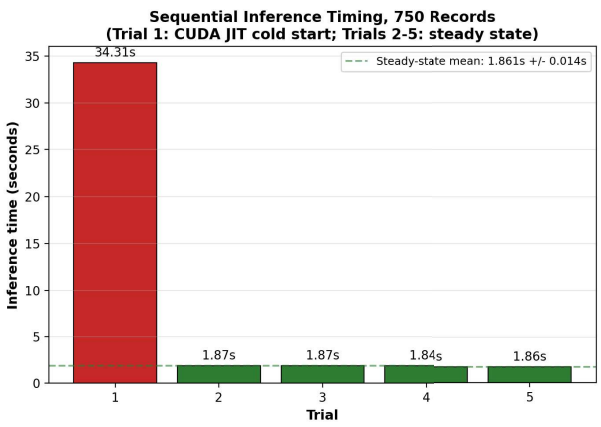


Fig. 2. Sequential inference timing across 5 repeated trials, 750 records. Trial 1's cold-start outlier is visually obvious against the tightly clustered steady-state trials 2–5.



Fig. 3. SLURM task execution timeline from real `sacct` data. Top: the 20-way array shows genuine concurrent pairs (a visible staircase of overlapping bars), consistent with the cluster's 2 physical GPUs. Bottom: the 2-way "persistent" configuration instead ran strictly sequentially, with task #1 starting exactly as task #0 ended the direct visual evidence for the unexplained concurrency loss discussed in Section V.

the sequential steady-state total of ∼3.0 s (model load plus inference for all 750 records in one process), the 20-way parallel configuration was **43.7× slower**.

2-way persistent-worker configuration. Reducing the shard count to 2 (matching the physical GPU count) was intended to test whether amortizing the fixed per-task cost over a larger per-worker share of data (375 records each) would recover a parallel speedup. It did not: total wall-clock was 25 s, still **8.3× slower** than sequential. Unexpectedly, `sacct` timestamps show the two tasks ran *sequentially* rather than concurrently in this run (task 1: 13 s; task 2, starting exactly as task 1 ended: 12 s), despite each requesting its own GPU on a cluster with 2 physical GPUs available in contrast to the genuine concurrency observed among pairs in the 20-way array run above. We cannot fully explain this discrepancy from the data available to us; the most plausible explanation is that `gpu001` is a shared node whose second GPU may have been in use by another user's job at the time, invisible to our own `squeue` query, though we did not independently confirm this and report it as an open question rather than a settled cause (Section VI). This should be read as an unresolved artifact of this run, not as evidence against two-GPU concurrency on this cluster in general, since genuine concurrency was observed among pairs in the 20-way array above.

Each individual task's own internally measured compute (model load plus inference) was only ∼2.2 s (Table IV) yet each task's `sacct`-reported wall-clock elapsed time was 12–13 s, leaving 10–11 s of overhead unaccounted for by anything our scripts measure. This gap is consistent, in both magnitude and direction, with what was observed per-task in the 20-way array (12–14 s per task there as well), strongly suggesting a fixed, per-SLURM-task startup cost most plausibly dominated by Python interpreter startup and `import torch`/`import transformers`, both well-documented to take several seconds rather than anything related to model size, GPU count, or shard size.

TABLE IV
PER-WORKER BREAKDOWN, 2-WAY PERSISTENT CONFIGURATION.

Worker	Records	Load (s)	Infer (s)
0	375	1.173	1.033
1	375	1.119	1.053

D. Summary Comparison

Table V and Figure 4 place all three configurations side by side. Every parallel configuration tested was slower than sequential execution for this workload, in stark contrast to the ∼6× speedup measured for the structurally analogous AWS Lambda/SQS design in our prior work [3], [4].

V. Discussion

The central finding of this study is not that decoupled parallel execution is a bad idea in general our own prior work [3], [4] measured real, substantial speedups from exactly this pattern but that the pattern's benefit is contingent on the cost structure of the execution substrate, and does not transfer automatically. Startup-overhead dominance for short jobs is not itself a new HPC observation; this paper's contribution is the direct, quantified comparison against a decoupled architecture that reversed our own prior, structurally analogous AWS speedup on this cross-substrate task [3], [4]. AWS Lambda's economic and technical model is built around cheap, fast, frequently-reused ("warm") invocations, where the marginal cost of adding another concurrent worker is small relative to the work each worker does. SLURM array jobs on a shared HPC cluster have a different cost structure: each array task is, from the operating system's perspective, closer to a fresh process launch, paying a fixed startup tax most plausibly library import time, though inferred rather than independently profiled of roughly 11–13 seconds regardless of how much or little work it then does. For a workload where per-task



Fig. 4. Total wall-clock duration for all three configurations, log scale, with slowdown factors relative to sequential execution annotated.

TABLE V
TOTAL WALL-CLOCK DURATION, 750-RECORD WORKLOAD, ALL CONFIGURATIONS.

Configuration	Wall-clock (s)	Relative to sequential
Sequential (steady-state)	∼3.0	1× (baseline)
2-way persistent array	25	8.3× slower
20-way array	131	43.7× slower

compute is small (a few seconds, as in our 750-record test set), that fixed tax dominates completely, and adding parallelism only multiplies how many times it gets paid.

This has a direct, testable implication that our results only partially explore: the break-even point should shift as the per-task workload grows relative to the fixed overhead. A shard large enough that its own inference time (at ∼400 records/sec, in the hundreds of seconds for tens of thousands of records) dwarfs an 11–13 s fixed cost should, in principle, show parallel speedup again. We did not test this directly (Section VI), and doing so is the most immediate and well-motivated next step for this line of work.

A. Cold-Start vs. Steady-State Baseline Sensitivity

The slowdowns above compare each SLURM configuration to the sequential pipeline's *steady-state* timing (∼3.0 s); each SLURM task, however, is itself a freshly launched process, more comparable to the sequential pipeline's own *cold-start* trial (≈35.5 s: ∼1.1 s load plus the 34.313 s cold inference trial, Table III). Against that baseline, the 20-way array's slowdown falls from 43.7× to ∼3.7×, and the 2-way configuration (25 s) is ∼1.4× *faster*. Both framings are legitimate steady-state for a long-running warm service, cold-start for a like-for-like comparison of independently launched processes and neither overturns the central finding, but the reported factor is baseline-dependent, and we recommend future work report both.

VI. LIMITATIONS

A. Template Diversity and Classification Generalizability

The perfect classification scores in Section IV-A most likely reflect the synthetic template generator's limited vocabulary and phrasing diversity (in particular, only four fixed hard-negative templates) rather than evidence that ClinicalBERT, fine-tuned this way, would perform this well on real, linguistically diverse clinical text. This is the single most important caveat on the classification results in this paper.

B. Single Shared GPU Node and Workload Scale

All parallel-execution experiments ran on Augie's single GPU-equipped node (`gpu001`, 2× A100), which may be shared with other users' jobs invisible to our own queue queries. This limits both the achievable degree of true parallelism and our ability to fully explain the concurrency behavior in Section IV-C. The comparison was also conducted at a single scale (750 test records); as discussed in Section V, fixed per-task overhead should matter proportionally less at larger scales, but we did not test this directly, and the conclusion that "parallel execution is slower here" should not be assumed to hold at all scales without further measurement. All experiments used a single HPC cluster and software stack, so these overhead magnitudes may not generalize to other SLURM configurations without replication.

C. Single Confirmatory Run at Larger Scale

The $n = 5{,}000$ classification result (Table II) reflects one seed, not the five-seed protocol used at $n = 500$, since the near-total ceiling effect at the smaller scale made further seeds unlikely to be informative; this is a reasoned trade-off, but it means the $n = 5{,}000$ result carries less statistical weight than the $n = 500$ result.

D. Infrastructure Reliability and Human Subjects

Downloading the `Bio_ClinicalBERT` checkpoint directly from the Hugging Face Hub proved unreliable from Augie during this study, requiring a manual download-and-transfer workaround; this does not affect the correctness of any reported result, since the same checkpoint file was used throughout. No real patient data of any kind was used in this study — all clinical notes are synthetic and programmatically generated, and no PHI, IRB approval, or data use agreement was required or implicated.

VII. FUTURE WORK

- **Highest priority:** repeat the comparison at the full 50,000-record scale already generated, to directly test the break-even argument in Section V.
- Independently profile `import torch`/`transformers` to confirm, rather than infer, the fixed-overhead mechanism.
- Investigate the unexplained 2-worker concurrency loss, ideally with cluster-administrator visibility into node-level GPU utilization.

- Test a persistent-daemon worker, the more direct SLURM-native analog of an AWS Lambda warm container.
- Replicate this comparison on at least one additional HPC/SLURM configuration to establish whether the reported overhead magnitudes are properties of SLURM-scheduled clusters generally or specific to Augie's configuration.
- Expand the synthetic generator's vocabulary and phrasing diversity, and extend to real clinical text (e.g., MIMIC-III) under credentialed access.

VIII. Conclusion

This paper reports a fully measured, honest empirical study of ClinicalBERT batch inference on Villanova University's Augie HPC cluster, using synthetic clinical text to avoid institutional PHI overhead. Classification performance was perfect across five seeds and a confirmatory larger-scale run a genuine but ceiling-bound result whose primary limitation, template diversity, we report explicitly. More centrally, decoupled SLURM-array parallel execution was substantially slower than sequential execution ($8.3\times$ to $43.7\times$, baseline-dependent per Section V-A), the opposite of the $\sim 6\times$ speedup our prior work measured for the structurally analogous AWS Lambda/SQS architecture. We identify the mechanism fixed per-task startup overhead of roughly 11–13 s dominating a workload whose actual compute is a few seconds and report an unexplained GPU-concurrency anomaly honestly rather than omit it. The methodological lesson is about architecture, not ClinicalBERT specifically: a pattern's performance benefit is a property of its interaction with a specific infrastructure's cost structure, not a portable property of the pattern itself, and claims of architectural speedup should be measured on the target infrastructure directly rather than assumed to transfer.

Acknowledgment

This work used the Augie High-Performance Computing cluster, funded by award NSF 2018933, at Villanova University.

Code and Data Availability

All source code used to produce every result, table, and figure in this paper the synthetic-data generator, deterministic splitter, fine-tuning script, sequential and SLURM-array inference benchmarks, SLURM batch scripts, `sacct` timing-capture script, and figure-generation script together with raw logs, manifests, and generated figures, is openly available at https://github.com/CMankala/clinicalbert-hpc-empirical-study. No real patient data is included or required; the synthetic dataset is generated deterministically from a fixed seed.

References

[1] E. Alsentzer, J. R. Murphy, W. Boag, W. H. Weng, D. Jin, T. Naumann, and M. McDermott, "Publicly available Clinical BERT embeddings," *arXiv preprint arXiv:1904.03323*, 2019.

[2] K. Huang, J. Altosaar, and R. Ranganath, "ClinicalBERT: Modeling clinical notes and predicting hospital readmission," *arXiv preprint arXiv:1904.05342*, 2019.

[3] C. K. Mankala and R. J. Silva, "Sustainable Real-Time NLP with Serverless Parallel Processing on AWS," *Information*, vol. 16, no. 10, p. 903, Oct. 2025. doi: 10.3390/info16100903.

[4] C. K. Mankala, "Unlocking the Potential of Cloud Computing for Sustainable Energy and Cost-Efficient Artificial Intelligence Workloads," in *Proc. 2025 IEEE Int. Workshop Metrology for Sustainability (MetroSustainability)*, Dec. 2025, doi: 10.1109/MetroSustainability67617.2025.11548174.

[5] H. Lee, S. S. Woo, and J. Yao, "Serverless parallel computing for data-intensive applications: An assessment of execution overhead," in *Proc. IEEE Int. Conf. Cloud Computing (CLOUD)*, San Francisco, CA, USA, 2018, pp. 312–319.

[6] S. Eismann, J. Grohmann, J. van Eyk, N. Herbst, and S. Kounev, "Serverless applications: Characterization and performance inspection," *IEEE Software*, vol. 38, no. 1, pp. 58–69, Jan.–Feb. 2021.

[7] L. Gu, D. Li, and H. Yu, "Breaking the concurrency limits: An event-driven asynchronous processing framework for serverless workflows," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 9, pp. 2241–2254, Sep. 2021.

[8] S. Kuhlenkamp and M. Werner, "Benchmarking serverless workflows: An evaluation of AWS Step Functions and Azure Logic Apps," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, Milan, Italy, 2019, pp. 244–251.

[9] Y. Zhang, M. Ju, and C. Li, "Clinical entity extraction and PHI de-identification via domain-specific BERT architectures," *JMIR Medical Informatics*, vol. 8, no. 8, p. e19432, Aug. 2020.

[10] H. Eykholt, T. A. Majchrzak, and S. Kuhlenkamp, "An empirical evaluation of serverless parallel processing backend scalability," in *Proc. IEEE Int. Conf. Cloud Eng. (ICCE)*, Orlando, FL, USA, 2018, pp. 112–118.